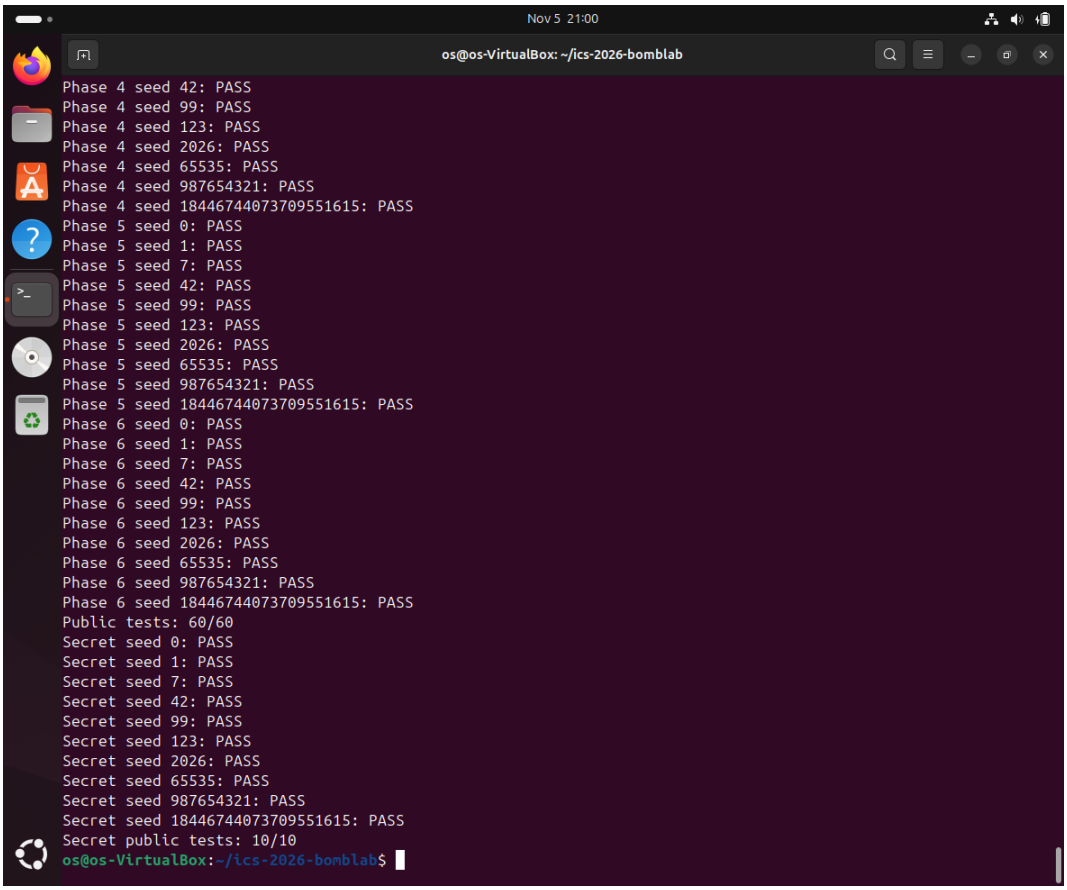
BombLab 实验报告

实验环境为 Ubuntu x86-64 虚拟机，使用 GCC 13.3.0、Make 4.3、Python 3.12.3、GDB 15.1 和 objdump 2.42。个人仓库为 https://github.com/Merlin0410/ics-2026-bomblab 。

本实验分析个人 bomb 的反汇编，在 defuser.c 中实现由 challenge 计算口令的函数。分析时使用 objdump 查看函数，使用 GDB 读取寄存器、跳转表、函数表和节点数据。以下涉及 32 位结果的加法、乘法均按模 2^32 计算，右移按无符号数处理。

六个正式关卡及 Honor 均完成。最终运行 make test，正式关卡公开测试为 60/60，Honor 为 10/10。另使用不在公开列表中的 seed 43 验证了六关和 Honor。公开测试结果不代表隐藏测试成绩。

本报告记录已完成的分析和验证，未把没有实际执行的调试步骤写成实验结果。



1. Phase 1：函数调用

phase_01 先检查一个无符号整数，再把输入与 p1_compute(challenge) 的返回值比较。0x4020db 调用 p1_compute，0x4020e0 的 cmp %eax,-0x14(%rbp) 检查结果是否相等。

p1_compute 中 shr $0x20,%rax 提取高 32 位，shr $0x10,%rdx 右移 16 位；调用顺序为 p1_step2、p1_step0、p1_step1。辅助函数中的 lea (%rdi,%rdi,2),%eax 表示乘 3。

记 lo=(uint32_t)c，hi=(uint32_t)(c>>32)，mid=(uint32_t)(c>>16)。恢复出的关系是：

a=(3*hi) ^ (lo+0xff82)

b=(mid+0x7bec) ^ lo

answer=(3*b+a) ^ 0x5384

注意 mid 是第 16～47 位，不是只取 16 位。用 uint32_t 实现，输出一个整数。

GDB 中在入口保存 challenge，再停在 0x4020e0。seed 42 的 challenge 为 0x28efe333b266f103，eax 与公式计算结果均为 3164644491。公开测试 10/10；执行 ./defuser --seed 43 --phase 1 | ./bomb --seed 43 --phase 1，输出 Phase 1 defused.。



2. Phase 2：数组与循环

valid_u32_fields 的参数为 6，sscanf 后也比较返回值是否为 6，说明需要六个整数。输入数组从 rbp-0x40 开始，每个元素占 4 字节。

0x402160～0x402169 的 shr、add、and、add 算出首项。循环中 lea (%rsi,%rsi,2),%ecx 计算前项的三倍；mov -0x40(%rbp,%rdx,4),%esi 读取当前项。下标从 1 增加到 6 时结束。

v[0]=(((uint32_t)(c>>9)*2)&0x3e)+5

v[i]=(3*v[i-1]) ^ (((c>>(16+i))&0x3f)+i)，i=1..5。

实现时先计算首项，再用循环生成后五项，count 设为 6。先加下标，再与前项三倍异或，不能改变运算顺序。

公开测试 10/10。额外执行 ./defuser --seed 43 --phase 2 | ./bomb --seed 43 --phase 2，challenge 为 0x6ec624aa85685867，输出 Phase 2 defused.。



3. Phase 3：跳转表

本关读取两个整数。and $0x7,%edi 得到 index=c&7；第一个输入必须等于 index。第二个参数 x=(uint32_t)(c>>3) ^ (uint32_t)(c>>32)，第二个输入须等于 p3_dispatch(index,x)。

关键指令为 jmp *0x405820(,%rax,8)。用 GDB 的 x/8gx 0x405820 读取表项，追踪地址后，index 0～7 对应的 case 函数编号为 7、0、6、2、1、5、3、4，不能按照汇编排列顺序判断。

各 index 的计算如下：

0: 5*x+0xca0a；1: 3*x+0xf59a；2: 5*x+0x1b503。

3: (x^0x4070)+(x>>1)；4: 5*x+0x5a9b。

5: (x^0xdf52)+(x>>2)；6: (2*x)^0xe6e0；7: (4*x)^0x27a7。

使用 switch 按表下标实现，输出 index 和计算结果。公开测试 10/10；seed 43 的 challenge 为 0xd49ffe504bfdab7f，分支为 7，通关成功。另外用临时脚本从 seed 0 开始选取不同分支进行验证，实际覆盖结果为 8/8。

额外验证命令：./defuser --seed 43 --phase 3 | ./bomb --seed 43 --phase 3。



4. Phase 4：递归

本关输入一个整数，与 p4_walk 的返回值比较。四个参数为 target=c&0x7f、low=0、high=127、key=(uint32_t)(c>>7) ^ (uint32_t)(c>>32)。

p4_walk 开头的 sub、shr $1、add 对应 mid=low+((high-low)>>1)。cmp %edi,%eax 后的 jb 表示 target 无符号小于 mid，进入左区间；否则进入右区间。递归时 key 不变。

target==mid：返回 (mid+0x98)^key。

target<mid：令 child=walk(target,low,mid-1,key)，返回 (3*child)^(mid+key)^0x0a。

target>mid：令 child=walk(target,mid+1,high,key)，返回 (3*child)^(mid+key)^0x06。

0x402380 的 lea 0x98(%rdi),%eax 和 xor %ecx,%eax 是终止分支；其余分支在 call 返回后乘 3 并异或。实现中保留递归结构，不能只返回找到目标时的值。

公开测试 10/10。额外执行 ./defuser --seed 43 --phase 4 | ./bomb --seed 43 --phase 4，challenge 为 0x006ff81b72d591ff，target=127，输出 Phase 4 defused.，验证了右端点路径。

```
os@os-VirtualBox:~/ics-2026-bomblab$ ./defuser --seed 43 --phase 4 | ./bomb --seed 43 --phase 4
BOMBLAB 2026                                    这里还没有明天

                        *
                         \
                      .-"""-.
                    .'       '.
                   /    LOOP    \
                  |               |
                   \             /
                    '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 当前] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x006ff81b72d591ff
Input (unsigned decimal integers):
Phase 4 defused.

[1 待解除] [2 待解除] [3 待解除] [4 已解除] [5 待解除] [6 待解除]
```

5. Phase 5：节点与指针

本关输入两个整数。p5_traverse 返回累计值，通过第二个指针参数写入最后处理的节点标识。mov (%rdx),%rdx 读取 next；mov 0x8(%rdx),%esi 读取标识；add 0xc(%rdx),%eax 读取数值。因此节点布局为 8 字节指针、4 字节标识、4 字节数值。

用 GDB 读取 0x405b60 的八个起点指针，以及 0x405ba0～0x405c5f 的节点。按地址递增编号 0～11，next 为 [3,9,11,8,-1,7,2,0,6,5,4,10]，起点表为 [1,9,5,7,0,3,8,6]。

标识（十六进制）：339,932,b59,72e,45,63f,12b,83b,a15,219,435,577。

数值（十六进制）：131f,2753,354b,380e,5e7a,2cba,e7cf,3254,ef79,1ff9,ddd9,7aa7。

起点为 heads[c&7]，处理节点数为 ((c>>3)&3)+1。初值 acc=(uint32_t)(c>>6)^(uint32_t)(c>>32)^0xa5d7。每个节点执行 acc=((acc+data)*5)^id，起始节点也要计算。输出最终 acc 和最后的 id。实现用下标代替原二进制中的指针。

公开测试 10/10。额外执行 ./defuser --seed 43 --phase 5 | ./bomb --seed 43 --phase 5，challenge 为 0x18e47d0995aecbfe，输出 Phase 5 defused.。

```
os@os-VirtualBox:~/ics-2026-bomblab$ ./defuser --seed 43 --phase 5 | ./bomb --seed 43 --phase 5
BOMBLAB 2026                                    这里还没有明天

                        *
                         \
                      .-""""-.
                    .'        '.
                   /    LOOP    \
                  |              |
                  \              /
                   '------------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 当前] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0x18e47d0995aecbfe
Input (unsigned decimal integers):
Phase 5 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 已解除] [6 待解除]
```

6. Phase 6：函数指针

本关输入一个整数。p6_dispatch 两次通过 0x405e00 的函数表间接调用。第一次 call *0x405e00(,%rax,8) 使用 i=c&7，第二次 call *0x405e00(%rbx) 的 rbx=c&0x38 已经是字节偏移，对应 j=(c>>3)&7。

用 x/8ga 0x405e00 读出的函数编号为 [3,4,0,5,1,0,4,2]。同一函数可能在表内重复出现。

op0(x)=ROL32(x,3)^0xad4e；op1(x)=ROR32(x,2)。

op2(x)=(x+0xf7d9)^(x>>3)；op3(x)=(x^0x9735)+4*x。

op4(x)=((x>>3)^x)+0x77db；op5(x)=(4*x+0x6a94)^(5*x)。

rol 和 ror 是循环移位，分别用 (x<<3)|(x>>29) 和 (x>>2)|(x<<30) 实现，x 使用 uint32_t。

初始 x=(uint32_t)(c>>6)^(uint32_t)(c>>32)。先算 a=F[i](x)，再算 b=F[j](a^0x8073)，最终输出 b+0x5d66。

公开测试 10/10。额外执行 ./defuser --seed 43 --phase 6 | ./bomb --seed 43 --phase 6，challenge 为 0x523fb4dd5d162e88，输出 Phase 6 defused.。

```
os@os-VirtualBox:~/ics-2026-bomblab$ ./defuser --seed 43 --phase 6 | ./bomb --seed 43 --phase 6
BOMBLAB 2026                                    这里还没有明天

                          *
                           \
                        .-""""-.
                      .'        '.
                     /    LOOP    \
                    |              |
                     \            /
                      '----------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 当前]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0x523fb4dd5d162e88
Input (unsigned decimal integers):
Phase 6 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 已解除]
```

7. Honor

secret_gate 用 strlen 后减 0x24，再与 0x7f 无符号比较，允许长度 36～163 字节。secret_copy 逐字节复制到 rbp-0xd0，包含结尾的零字节。随后 cmpl $0x4e45504f,-0xb0(%rbp) 检查偏移 32 的标记。小端序下标记对应 OPEN。rbp-0xac 即偏移 36，作为 secret_phase 的输入。

因此构造 32 个 A、OPEN、三个空格分隔的无符号整数。这里涉及相邻字段覆盖，没有覆盖返回地址。secret_phase 检查三个整数，secret_mix(x)=((x>>7)^0xc9df)+(x^0xbdb2)。

令 a=(uint32_t)c^(uint32_t)(c>>32)，b=(uint32_t)context^(uint32_t)(context>>32)。三个整数为：

first=context&3

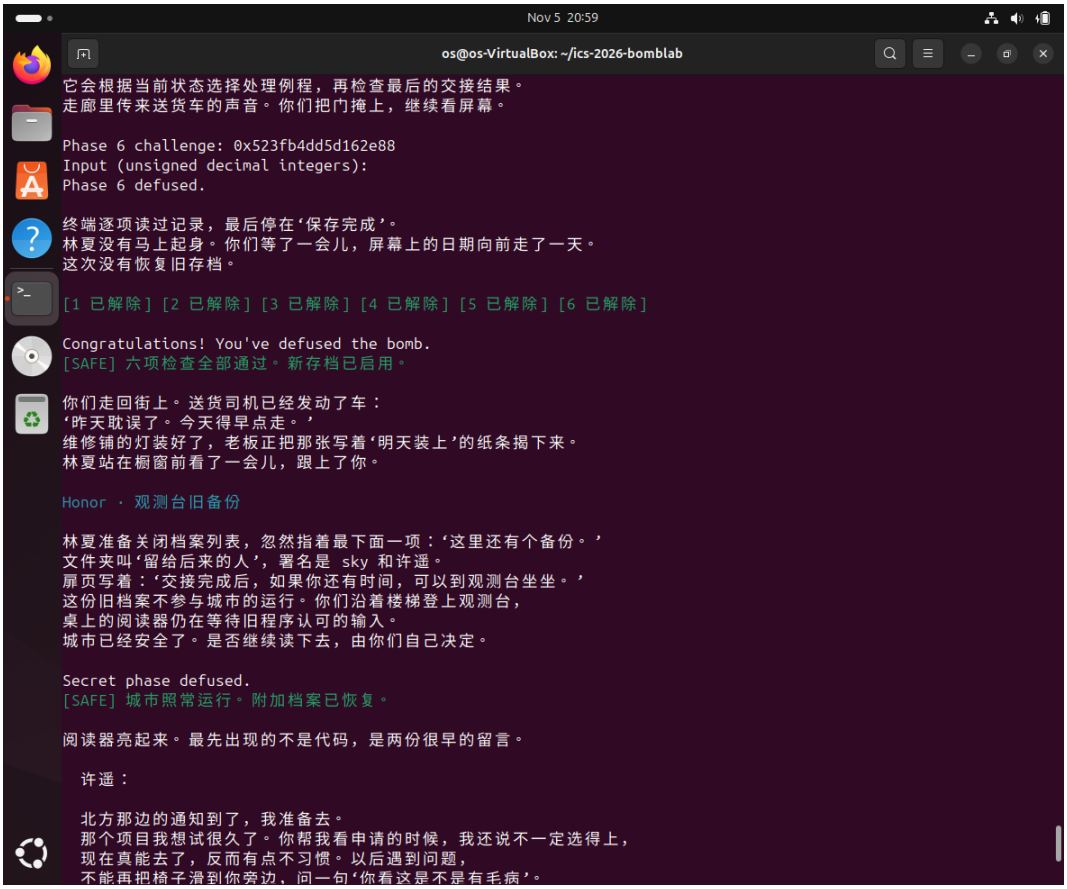
second=secret_mix(a^b)^((first*0xbdb2)^(uint32_t)(context>>16))

third=second^(b+first+0xc9df+(a>>7))

所有中间结果采用 uint32_t。使用 snprintf 拼接第七行，检查容量，不在 out 内添加换行。context 由驱动根据前六关口令计算，不能固定为某个 seed 的值。

python3 public_test.py --secret 得到 10/10。额外执行下面命令，seed 43 的六关和 Honor 均通过：

{ ./defuser --seed 43; ./defuser --secret --seed 43; } | ./bomb --seed 43



8. 验证记录与参考资料

正式六关分别运行 python3 public_test.py --phase N，均为 10/10。每关也分别使用 seed 43 执行 ./defuser --seed 43 --phase N | ./bomb --seed 43 --phase N，均输出 Phase N defused.。这里 N 为 1～6。

Honor 的公开测试为 10/10；最终 make test 显示正式六关 60/60、Honor 10/10。第三关额外检查了八个分支，覆盖 8/8。其余关卡没有声称穷举全部输入。

实验过程中曾在编辑 defuser.c 后出现 uint64_t 和 size_t 无法识别的编译错误，提示需要保留模板头部和辅助函数。后续实现通过了编译及完整测试。修改单个函数时，应避免删除其他函数或文件头部。

参考资料：课程 Lab2 BombLab 实验说明，https://ics-26fall-fdu.github.io/labs/lab2-bomb-lab/ 。实际分析依据为个人 bomb 的 objdump 输出、GDB 观察结果以及仓库模板接口和公开测试。

辅助工具说明：本实验使用 AI 辅助理解汇编、整理求解代码及排查错误，并在本地进行调试、公开测试和额外 seed 验证。报告依据实际输出整理。

```
os@os-VirtualBox:~/ics-2026-bomblab$ python3 public_test.py --secret
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```